

String Processing Macros (histrings.dtx)

Charles Duan

September 6, 2026

This package provides a variety of low-level text processing commands and macro helpers. The section in the main *Hereinafter* documentation just provides an overview of features; the full documentation for this file should be consulted for the particular macros and their usage.

Initially, the package provides several syntactic sugar macros for conditionals, definitions, and expansion control. These are similar to the `etoolbox` package.

```
\ProvidesPackage{histrings}[2004/10/09 String macros and other utilities]
\RequirePackage{etoolbox}
```

1 Conditionals

Performs an `.`-type test. Takes an argument of an if-conditional, like:

```
\ifx#1\test
```

and turns it into:

```
\ifx#1\test
  \expandafter\@firstoftwo
\else
  \expandafter\@secondoftwo
```

Usage is like:

```
\@test\if(test)\fi{<true stuff>}{<false stuff>}
```

The delimiting ensures that the test will be properly skipped over inside another conditional.

```
\def\@test#1\fi{#1\expandafter\@firstoftwo\else\expandafter\@secondoftwo\fi}
```

Performs a series of `\@test` cases in the form

```
\@testcase
  \if...\fi{Do this}%
  \if...\fi{Do this}%
  \if...\fi{Do this}%
  \default{Do this otherwise}%

\def\@testcase#1\fi#2#3\default#4{%
  \@test#1\fi{#2}{\ifstrempy{#3}{#4}{\@testcase#3\default{#4}}}%
}
```

2 Expansion Control

Although there are several useful macros defined in this section, the notable one is `\@expand`, which expands a token multiple times. It takes three arguments: the callback to be executed when done expanding, the stuff to expand, and a count of how many expansions are to be performed, represented by the number of tokens in the argument. Thus:

```
\def\macro{Hello World}
\@expand{\process}{\csname macro\endcsname}{ii}
```

would expand `\csname macro\endcsname` twice (once for each “i”), obtaining the meaning of `\macro`, and then run the callback `\process`, to result in:

```
\process{Hello World}
```

```
\long\def\@expand#1#2#3{\@expand{#2}{#1}#3\@stop}
\long\def\@expand#1#2#3#4\@stop{%
  \ifstrempy{#4}{%
    \expandafter\@expand\expandafter{#1}{#2}%
  }{%
    \expandafter\@expand\expandafter{#1}{#2}#4\@stop
  }%
}
\long\def\@expand#1#2{#2{#1}}
```

Returns the first of three groups.

```
\def\@firstofthree#1#2#3{#1}
```

`\@expandarg`

Expands #2 once and provides it as an argument to #1 (which must be a single token macro).

```
\def\@expandarg#1#2{\expandafter#1\expandafter{#2}}%
```

Expands #2 and #3 once, and runs #1 with the expansions as arguments.

```
\def\@expandtwo#1#2#3{%
  \expandafter\@expandtwo\expandafter{#2}{#3}{#1}%
}
\def\@expandtwo#1#2#3{%
  \expandafter\@expandtwo\expandafter{#2}{#1}{#3}%
}
\def\@expandtwo#1#2#3{%
  #3{#2}{#1}%
}
```

These macros are tokens used to delimit parts of macro arguments.

```
\let\str@mark\relax
\let\str@stop\relax
\def\str@test{\str@stop}
```

Removes braces from the arguments. This is useful if you have a callback that takes a series of arguments, and you use one of the functions like `\@expand` that puts braces around the callback result.

```
\def\@unbrace#1#2{#1#2}
```

A useful helper for `\find@in`: takes two macro names and two definitions, and assigns the macro names to the definitions.

```
\def\@twodef#1#2#3#4{%
  \def#1{#3}\def#2{#4}%
}
```

Takes two callbacks and two parameters, and runs each callback on the corresponding parameter.

```
\def\@tworun#1#2#3#4{%
  #1{#3}#2{#4}%
}
%
% Takes one callback and two parameters, discards the second parameter, and runs
```

```
% the callback on the first parameter. (The alternative, where the first
% parameter is discarded and the callback is run on the second parameter, can be
% achieved with |\tworun@gobble|\margin{callback}.)
%
% \begin{macrocode}
\def\rungobble#1#2#3{%
  #1{#2}%
}
```

Pads a string #1 with spaces so that its length equals the length of string #2. Fully expandable.

```
\def\pad@string#1#2{%
  \pad@string@#1\str@mark#2\str@stop}%
}
\def\pad@string@#1#2\str@mark#3#4\str@stop#5{%
  \ifstrempy{#4}{%
    #5#1#2%
  }{%
    \ifstrempy{#2}{%
      \pad@string@{ }%
    }{%
      \pad@string@#2%
    }%
  }%
}
```

3 Adding to Macros

These macros could all be replaced with `etoolbox` command equivalents.

Adds to a macro.

```
\def\addto@macro#1#2{\expandafter\def\expandafter#1\expandafter{#1#2}}
```

Adds to a macro, but instantiates the macro as empty if it didn't exist before.

```
\def\addto@list#1#2{%
  \ifundefined{\expandafter\@gobble\string#1}{%
    \def#1{#2}%
  }{\addto@macro#1{#2}}%
}
```

Prepends to a macro.

```
\def\prepend@macro#1#2{%
  \expandafter\prepend@macro\expandafter{#1}{#1}{#2}%
}
\long\def\prepend@macro#1#2#3{\def#2{#3#1}}
```

Adds the contents of a macro to a macro.

```
\def\add@macro@to@macro#1#2{%
  \expandafter\addto@macro\expandafter#1\expandafter{#2}%
}
```

If the list is undefined or empty, adds the given element. Otherwise, adds a space and the element.

```
\def\spcapppto#1#2{%
  \ifx#1\relax \def#1{#2}\else
    \ifx#1\empty \def#1{#2}\else
      \appto#1{ #2}%
    \fi
  \fi
}
```

4 Finders

These macros provide convenient ways of defining text finders that, when called, are fully expandable. They provide the ability to find text *<needle>* at the beginning, middle, or end of text *<haystack>*. The pattern is to call:

```
\make@find@<type>{<needle>}
```

to initialize the finder, and then:

```
\find@⟨type⟩{⟨needle⟩}{⟨haystack⟩}
  {⟨true-callback⟩}{⟨false-callback⟩}
```

to execute the finder. $\langle type \rangle$ is in, start, end, or eq depending on the type of matching desired.

The $\langle needle \rangle$ may contain control sequences, as it is passed through $\detokenize$ appropriately in constructing and using finders.

The callbacks for finders are given consistent arguments. The $\langle false-callback \rangle$ always receives no arguments. The $\langle true-callback \rangle$, however, receives arguments indicating the remaining unmatched text. Thus, for $\backslash find@in$, the $\langle true-callback \rangle$ receives two arguments, the prefix and suffix relative to the matched text. However, the callbacks for $\backslash find@start$ and $\backslash find@end$ receive only one argument, and $\backslash find@eq$'s callback receives none.

The design of finders is as follows. To make a finder, a macro is defined with an argument template to match the text to be found. That macro is called with the text to be searched (with some additional trimmings to help determine whether the finder macro matched).

This is a helper command that other finders use, which invokes the internal finding macro.

```
\def\find@use#1#2#3{%
  \ifundefined{finder@#1\detokenize{#2}}{%
    \PackageError{histrings}{%
      Finder of type #1 for '\detokenize{#2}' not defined; used%
    }{%
      You must run \@backslashchar make@find@#1\detokenize{#2} first%
    }\@gobbletwo
  }{%
    \nameuse{finder@#1\detokenize{#2}}#3%
  }%
}
```

$\backslash make@find@in$

Makes a finder that searches for text anywhere within a string.
The actual finder macro takes input of the form:

```
\finder@in@⟨text⟩
  ⟨haystack⟩\str@mark
  ⟨needle⟩\str@mark\str@stop
  {⟨text-to-search⟩}
  {⟨callback-if-found⟩}
  {⟨callback-if-not-found⟩}
```

```
\def\make@find@in#1{%
  \ifundefined{finder@in\detokenize{#1}}{%
    \global\@namedef{%
      finder@in\detokenize{#1}%
    }##1##2\str@mark##3\str@stop##4##5{%
      \ifstrempy{##3}{##5}{##4{##1}{##2}}%
    }%
  }{%
  }
```

$\backslash find@in$

Finds text using a predefined finder. Takes four arguments: the text to find, the text to be searched, the callback if #1 is found, and the callback if #1 is not found. The first callback takes two arguments, which are the pre-match text and post-match text; the latter callback takes no arguments.

```
\def\find@in#1#2{%
  \find@use{in}{#1}{#2\str@mark#1\str@mark\str@stop}%
}
```

Like `\find@in`, but finds the last instance of #1 in #2 rather than the first. Because this has to iterate against all of #2, this will likely mess up internal grouping in #2.

```
\def\find@last#1#2{%
  \find@in{#1}{#2}{%
    \find@last@{#1}%
  }{\@secondoftwo}%
}
\def\find@last@#1#2#3{%
  \find@in{#1}{#3}{\find@last@{#1}{#2}}{\find@last@@{#2}{#3}}%
}
\def\find@last@@#1#2#3#4{%
  \find@in{#1}{#4}{\find@last@{#1}{#2#1#3}}{\find@last@@{#2#1#3}{#4}}%
}
\def\find@last@@@#1#2#3#4{#3{#1}{#2}}
```

Makes a named finder with name #1, so #2 can contain control sequence names.

```
\def\make@find@in@cs#1#2{%
  %
  % The actual finder macro takes input of the form:
  % \finder@in@[text]
  % [text-to-search]\str@mark
  % [text-to-find]\str@mark\str@stop
  % {[text-to-search]}{[callback-if-found]}{[callback-if-not-found]}
  \@ifundefined{finder@cs@in@#1}{%
    \global\@namedef{finder@cs@in@#1}##1#2##2\str@mark##3\str@stop##4##5{%
      \ifstrepty{##3}{##5}{##4{##1}{#2}}%
    }%
    \global\@namedef{findcs@in@#1}##1{%
      \@nameuse{finder@cs@in@#1}##1\str@mark##2\str@mark\str@stop
    }%
  }{}%
}
```

The equivalent of `\find@in` except for command sequences.

```
\def\find@in@cs#1#2{%
  \@nameuse{findcs@in@#1}{#2}%
}
```

Makes finders that require the two strings to be equal.

```
\def\make@find@eq#1{%
  \@ifundefined{finder@eq@\detokenize{#1}}{%
    %
    % This finder expects to be called as:
    % \finder@eq@[text]
    % [text-to-compare]\str@mark
    % [text-of-finder]\str@mark\str@stop
    % {macro-if-yes}{macro-if-no}
    % Both macros take no arguments.
    \global\@namedef{%
      finder@eq@\detokenize{#1}%
    }##1#1##2\str@mark##3\str@stop##4##5{%
      \ifstrepty{##1##2}{##4}{##5}%
    }%
  }{}%
}
```

Tests if the two strings are equal. Takes four arguments: the two strings to compare, a macro if they are equal, and a macro if they are not (the two macros taking no arguments).

```
\def\find@eq#1#2{%
  \find@use{eq}{#1}{#2\str@mark#1\str@mark\str@stop}%
}
```

Makes a finder that finds a text at the end of a string.

```
\def\make@find@end#1{%
  \@ifundefined{finder@end@\detokenize{#1}}{%
    \global\@namedef{%
      finder@end@\detokenize{#1}%
    }##1#1\str@mark##2\str@stop##3##4{%
      \ifstrepty{##2}{##4}{##3{##1}}%
    }%
  }{}%
}
```

Finds text at the end of a string. Takes four arguments, the first two being the string to find and the string to be searched. Upon a match, execute #3 with one argument, the front matter of the string. Otherwise, execute #4 with no argument.

```
\def\find@end#1#2{%
  \find@use{end}{#1}{#2\str@mark#1\str@mark\str@stop}%
}
```

Makes a finder that finds a text at the start of a string.

```
\def\make@find@start#1{%
  \@ifundefined{finder@start@detokenize{#1}}{%
    \global\@namedef{%
      finder@start@detokenize{#1}%
    }##1\str@mark#1#2\str@mark#3\str@stop##4##5{%
      \ifstrempy{##1}{##4{##2}}{##5}%
    }%
  }{%
  }%
}
```

Finds text at the start of a string. Takes four arguments, the first two being the string to find and the string to be searched. Upon a match, execute #3 with one argument, the trailing matter of the string. Otherwise, execute #4 with no argument.

```
\def\find@start#1#2{%
  \find@use{start}{#1}{\str@mark#2\str@mark#1\str@mark\str@stop}%
}
```

Defines a command sequence start finder. #1 is a nickname for the finder, #2 the sequence to search for.

```
\def\make@find@start@cs#1#2{%
  \@ifundefined{finder@cs@start@#1}{%
    \global\@namedef{%
      finder@cs@start@#1%
    }##1\str@mark#2##2\str@mark#3\str@stop##4##5{%
      \ifstrempy{##1}{##4{##2}}{##5}%
    }%
    \global\@namedef{findcs@start@#1}##1{%
      \@nameuse{finder@cs@start@#1}%
      \str@mark#1\str@mark#2\str@mark\str@stop
    }%
  }{%
  }%
}\def\find@start@cs#1#2{\@nameuse{findcs@start@#1}{#2}}
```

Iterates over a series of possible finders, running a function upon success. The arguments are:

- #1 is the find command (\find@in, \find@start etc.)
- #2 is a list of pairs of parameters as explained below.
- #3 is the text to be searched
- #4 is a callback if none of the matches succeed. It should take no arguments.

The parameter pairs argument looks as follows:

```
{%
  {<needle-1>}{<true-callback-1>}
  {<needle-2>}{<true-callback-2>}
  {<needle-3>}{<true-callback-3>}
  ...
}
```

The callbacks receive the number of arguments corresponding to the finder command (#1).

```
\def\find@try#1#2#3#4{%
  \find@try{#1}#2\str@stop{#3}{#4}%
}
% This is a table mapping arguments to this \find@try@ helper.
%\find@try \find@try@
% #1      #1
% #2, pair #2, #3
% #2, rest #4
% #3      #5
% #4      #6
```

```

\def\find@try@#1#2#3#4\str@stop#5#6{%
  #1{#2}{#5}{#3}{%
    \ifstrempy{#4}{#6}{\find@try@{#1}#4\str@stop{#5}{#6}}%
  }%
}

```

Chops space off of a string. Fully expandable.

```

\newcommand*\chop@space@then@run[2]{%
  \find@start{ }#1{\chop@space@then@run}{\str@chopspc@end{#1}}{#2}%
}
\def\str@chopspc@end#1#2{%
  \find@end{ }#1{\str@chopspc@end}{\str@chopspc@run{#1}}{#2}%
}
\def\str@chopspc@run#1#2{#2{#1}}
\newcommand*\chop@space[1]{\chop@space@then@run{#1}{\@iden}}
\make@find@start{ }
\make@find@end{ }

```

5 Cardinal Numbers

\cardinaltext The `\cardinaltext` macro changes a number into its capitalized, cardinal textual form. For example, “123” becomes “One Hundred Twenty-Three”.

```

\def\cardinaltext#1{%
  \ifnum#1<100
    \cardinaltext@digits#1@stop
  \else
    #1
  \fi
}
\def\cardinaltext@digits#1#2@stop{%
  \ifstrempy{#2}{%
    \cardinaltext@ones{#1}%
  }{%
    \ifnum#1=\@ne
      \cardinaltext@teens{#2}%
    \else
      \cardinaltext@tens{#1}%
      \ifnum#2>\z@
        -\cardinaltext@ones{#2}%
      \fi
    \fi
  }%
}
\def\cardinaltext@ones#1{%
  \ifcase#1Zero\or One\or Two\or Three\or Four\or Five\or Six\or Seven\or
  Eight\or Nine\fi
}
\def\cardinaltext@teens#1{%
  \ifcase#1Ten\or Eleven\or Twelve\or Thirteen\or Fourteen\or Fifteen\or
  Sixteen\or Seventeen\or Eighteen\or Nineteen\fi
}
\def\cardinaltext@tens#1{%
  \ifcase#1\or\or Twenty\or Thirty\or Forty\or Fifty\or Sixty\or Seventy\or
  Eighty\or Ninety\fi
}

```

6 Text Tokenizer

Often it is useful to tokenize a string of $\text{\TeX}$ tokens into words, in a way that properly handles macros and groups, and differentiates the tokens by type. The following macros help to do that.

\find@next The `\find@next` macro finds the next single token by type and runs a command on it. This macro can detect spaces, groups, and characters at the beginning of a string. Its usage is as follows:

```
\find@next{<str>}{<space>}{<group>}{<char>}{<end>}
<str>:    the string to read
<space>:  callback for a space
<group>:  callback for a group
<char>:   callback for a character
<end>:    callback if the string is empty
```

The `<space>`, `<group>`, and `<char>` callbacks each take two arguments, the found item and the remainder of the string. The `<end>` callback receives no arguments.

#1 is the string to test, #2 is the callback for spaces, #3 for groups, #4 for characters, #5 for when there is nothing left.

XXX: These functions do not appear to handle groups at the ends of strings correctly.

```
def\find@next#1#2#3#4#5{%
  \find@start{ }{#1}{#2{ }}{%
    \ifstrempy{#1}{#5}{%
      \find@next@group#1\str@mark{}\str@stop{#3}{#4}%
    }%
  }%
}
def\find@next@group#1#1{%
  \ifstrempy{#1}\find@next@group@yes{\find@next@group@no#1}%
}
def\find@next@group@yes#1#2\str@mark#3\str@stop#4#5{#4{#1}{#2}}
def\find@next@group@no#1#2\str@mark#3\str@stop#4#5{#5{#1}{#2}}
}
#5{#1}{#2}%
}
```

`\find@word` Finds the next word in the given string and runs a callback on it. This is like `\find@next` except (1) it aggregates letters into a single word, and (2) it differentiates letters from other characters. Its usage is as follows:

```
\find@word{<str>}{<sym>}{<group>}{<word>}{<end>}
<str>:    the string to read
<sym>:    callback for a symbol (space or non-letter character)
<group>:  callback for a group
<word>:   callback for a word
<end>:    callback if the string is empty
```

#1 is the string to test, #2 is the callback for non-letter characters (including spaces), #3 for groups, #4 for words, #5 for when there is nothing left.

```
def\find@word#1#2#3#4#5{%
  \find@next{#1}{#2}{#3}{\find@word@char{#2}{#4}}{#5}%
}
}
```

This is called if the first character of the string is a character. Tests if it's a letter. If so, then runs `\find@word@next` on the next character, specifying that (1) if the subsequent character is also a letter then run `\find@word@acc` to continue accumulating a word, but (2) if the subsequent character is not a letter then run the word callback.

#1 is the punctuation callback; #2 is the word callback; #3 is the character most recently read; #4 is the remainder of the text.

```
def\find@word@char#1#2#3#4{%
  \ifwordchar{#3}{%
    \find@word@next{#4}{#2{#3}{#4}}{\find@word@acc{#2}{#3}{#4}}%
  }{%
    #1{#3}{#4}% First char of the string was punctuation
  }%
}
}
```


Runs `\find@next` on #1, calling #2 with no args on anything other than a character and #3 with args on a character.

```
\def\find@word@next#1#2#3{%
  \find@next{#1}%
  {\@firstofthree{#2}}{\@firstofthree{#2}}{#3}{#2}%
}
```

Knowing that at least one word character has been found, continues reading the text to find all the word characters, calling itself recursively to do so. #1 is the word callback; #2 is the accumulated word so far; #3 is the full remaining text after #2; #4 is the first character of #3, and #5 is the text after the first character of #3.

```
\def\find@word@acc#1#2#3#4#5{%
  \@ifwordchar{#4}{%
    \find@word@next{#5}{#1{#2#4}{#5}}{\find@word@acc{#1}{#2#4}{#5}}%
  }{%
    #1{#2}{#3}%
  }%
}
```

7 Searching for Braced Groups

`\find@group` The `\find@group` macro finds a braced group in a string and runs a callback with three arguments: the pre-group text, the group (without braces), and the post-group text.

`\find@group@end` The `\find@group@end` macro similarly finds a braced group, but only at the end of a string. The callback receives only two arguments.

#1 is the text, #2 the callback, and #3 a callback if no braced group is found.

```
\def\find@group#1#2#3{%
  \find@group@#1\str@mark{.}\str@stop{#2}{#3}%
}
\def\find@group@#1{\find@group@{#1}}
```

#1 is the pre-group text, #2 the group, #3 the post-group text (possibly with `\str@mark`), #4 and #5 callbacks.

```
\def\find@group@@#1#2#3\str@stop#4#5{%
  \ifstrempy{#3}{#5}{\find@group@yes{#1}{#2}{#3}\str@stop{#4}}%
}
```

#1-#3 are the pre-, in-, and post-group text, #4 the empty material, #5 the callback.

```
\def\find@group@yes#1#2#3\str@mark#4\str@stop#5{#5{#1}{#2}{#3}}
```

Finds a braced group at the end of a text. Runs a callback with two arguments, the pre-group text and the group. #1 is the text, #2 the callback with a group, and #3 the callback if no group.

```
\def\find@group@end#1#2#3{%
  \find@group@end@{#1}{#2}{#3}%
}
```

#1 is the already-processed text, #2 the text remaining to process, #3 and #4 the callbacks.

```
\def\find@group@end@#1#2#3#4{%
  \find@group{#2}{%
    \find@group@end@@{#1}{#3}{#4}%
  }{#4}%
}
```

#1 is the already-processed text, #2-3 are the callbacks; #4-#6 the pre-, in-, and post-group text.

```
\def\find@group@end@@#1#2#3#4#5#6{%
  \ifstrempy{#6}{#2{#1#4}{#5}}{%
    \find@group@end@{#1#4}{#5}{#6}{#2}{#3}%
  }%
}
```

8 Testing Text Types

Tests if something is a character.

```
\def\ifletter#1{\@test\ifcat a\@noexpand#1\fi}
```

Tests whether a string is a single character. If it is zero characters, this function treats it as false. A macro will count as a single character.

```
\def\ifonechar#1{\ifstrempy{#1}{\@secondoftwo}{\@ifonechar@#1\str@stop}}
\def\@ifonechar@#1#2\str@stop{\ifstrempy{#2}}
```

Tests if something is a single digit token.

```
\def\ifdigit#1{\@ifonechar{#1}{\@test\ifnum 9<1\@noexpand#1 \fi}{\@secondoftwo}}
```

Tests if something is a word character (letter or number).

```
\def\@ifwordchar#1{%
  \ifletter{#1}\@firstoftwo{\@ifdigit{#1}\@firstoftwo\@secondoftwo}%
}
```

Tests if something is a number. A negative number is permitted.

```
\def\@ifnumber#1{%
  \ifblank{#1}\@secondoftwo{%
    \find@start{-}{#1}{\@unbrace\@ifnumber@}{%
      \@ifnumber@#1%
    }\@stop
  }%
}
\make@find@start{-}
\def\@ifnumber@#1#2\@stop{%
  \ifdigit{#1}{%
    \ifblank{#2}\@firstoftwo{\@ifnumber@#2\@stop}%
  }\@secondoftwo
}
```

Tests whether a string ends with a letter.

```
\def\@ifendswithwordchar#1{%
  \ifstrempy{#1}{\@secondoftwo}{%
    \find@end{ }{#1}{\@thirdofthree}{%
      \str@iffewwc#1\str@stop
    }%
  }%
}
\def\str@iffewwc#1#2\str@stop{%
  \ifstrempy{#2}{%
    \@ifonechar{#1}{%
      \@ifwordchar{#1}%
    }{%
      \str@iffewwc#1\str@stop
    }%
  }{%
    \str@iffewwc#2\str@stop
  }%
}
```

Tests whether a string starts with a letter.

```
\def\@ifstartswithwordchar#1{%
  \ifstrempy{#1}{\@secondoftwo}{%
    \find@start{ }{#1}{\@thirdofthree}{%
      \@ifonechar{#1}{%
        \@ifwordchar{#1}%
      }{%
        \expandafter\@ifstartswithwordchar\expandafter{\@car#1\@nil}%
      }%
    }%
  }%
}
```

1 Advanced List Processing

This extends the `etoolbox` macro for internal lists, enabling insertion of separators depending on the length of the list.

`\sepforslistloop` $\langle handler \rangle$ $\langle 2sep \rangle$ $\langle 3mid \rangle$ $\langle 3last \rangle$ $\langle list \rangle$

Performs $\langle handler \rangle$ for each element of $\langle list \rangle$. If there are two elements in the list, then performs $\langle 2sep \rangle$ before the second element. If there are three or more elements, then performs $\langle 3mid \rangle$ between all elements except the last, and performs $\langle 3last \rangle$ before the last element.

```
\def\sepforslistloop#1#2#3#4#5{%
  \test\ifx#5\relax\fi}%
  \test\ifx#5@empty\fi}%
  \expandafter\str@forlistloop@i#5\str@stop{#1}{#2}{#3}{#4}%
}%
}
```

The remaining macros are analogous to those in `etoolbox`, but in each case, the $\langle 2sep \rangle$, $\langle 3mid \rangle$, and $\langle 3last \rangle$ arguments come immediately before the list name.

`\sepdolistloop` $\langle 2sep \rangle$ $\langle 3mid \rangle$ $\langle 3last \rangle$ $\langle list \rangle$

Executes the macro `\do` on each element of the list and adds separators.

`\sepforslistcsloop` $\langle handler \rangle$ $\langle 2sep \rangle$ $\langle 3mid \rangle$ $\langle 3last \rangle$ $\langle listcs \rangle$

Executes the $\langle handler \rangle$ on each element of the list in macro $\langle listcs \rangle$ and adds separators.

`\sepdolistcsloop` $\langle 2sep \rangle$ $\langle 3mid \rangle$ $\langle 3last \rangle$ $\langle listcs \rangle$

Executes the macro `\do` on each element of the list in macro $\langle listcs \rangle$ and adds separators.

```
\def\sepdolistloop{\sepforslistloop\do}
\def\sepforslistcsloop#1#2#3#4#5{%
  \expandafter\sepforslistloop{#1}{#2}{#3}{#4}{\csname #5\endcsname}{i}%
}%
\def\sepdolistcsloop#1#2#3#4{%
  \expandafter\sepdolistloop{#1}{#2}{#3}{\csname #4\endcsname}{i}%
}%
}
```

`etoolbox` uses a specially-catcoded pipe symbol as its list delimiter. The `\str@reset@catcodes` macro retains the original catcode for the pipe symbol, so it is executed after all the list processing macros to restore the catcode.

```
\edef\str@reset@catcodes{\catcode\number'\|=\the\catcode'\|\relax}
\catcode'\|=3
```

Each of these macros separate off one list element and parse arguments as follows: #1 the next list element, #2 the remainder of the list, #3 the per-item handler, #4 the 2-element separator, #5 the 3-element non-last separator, #6 the 3-element last separator. The macro ending in `@i` acts on the first element of the list, `@i i` on the second element, and `@i i i` on the third and further elements. Thus, a list with only one element stops at `@i`, a list with only two elements stops at `@i i`, and a list with three or more elements will have #2 empty on its last element.

```
\long\def\str@forlistloop@i#1|#2\str@stop#3#4#5#6{%
  #3{#1}%
  \ifstrempy{#2}{}%
    \str@forlistloop@i#2\str@stop{#3}{#4}{#5}{#6}%
  }%
```

```

}
\long\def\str@forlistloop@iii#1|#2\str@stop#3#4#5#6{%
  \ifstrempy{#2}{%
    #4#3{#1}%
  }{%
    #5#3{#1}%
    \str@forlistloop@iii#2\str@stop{#3}{#4}{#5}{#6}%
  }%
}
\long\def\str@forlistloop@iii#1|#2\str@stop#3#4#5#6{%
  \ifstrempy{#2}{%
    #6#3{#1}%
  }{%
    #5#3{#1}%
    \str@forlistloop@iii#2\str@stop{#3}{#4}{#5}{#6}%
  }%
}
\str@reset@catcodes

```